

Account, PersonalAcct, BusinessAcct ReflectionLog

Understanding the Problem

I understood the problem as creating a base Account class and then extending it into PersonalAccount and BusinessAccount, each with their own withdrawal rules and fees.

At first, I was confused about how multiple classes worked together, especially how Customer was used inside Account. I resolved this by realizing it's an example of **composition** (an object inside another object).

Code reference (Account class with Customer):

```
6 {  
7     private double balance;  
8     private Customer cust;  
9     private String acctID;
```

Planning the Solution

I broke the problem into parts: create the Account class, add methods like deposit and withdrawal, then extend it into specialized accounts with extra rules.

I chose inheritance so both account types could reuse the same balance and ID logic.

Code reference (inheritance):

```
3 public class PersonalAccount extends Account  
3 public class BusinessAccount extends Account
```

Implementation

I first created the Account class with basic features like deposit, withdrawal, and toString(). Then I added the subclasses and overrode the withdrawal() method to include fees. Finally, I tested everything in BankTest.

I tested by printing account info before and after withdrawals.

Code reference (test class):

```
31-         if (choice == 1)
32-         {
33-             acct = new PersonalAccount(balance, fName, lName);
34-         }
35-         else if (choice == 2)
36-         {
37-             acct = new BusinessAccount(balance, fName, lName);
38-         }
```

Overcoming Challenges

The hardest part was overriding the `withdrawal()` method and making sure the fee only applied when the balance dropped below the minimum. I also had to understand when to use `super.withdrawal()`.

I solved this by carefully testing different withdrawal amounts.

Code reference (override method):

```
18         @Override
19-         public void withdrawal(double amt)
20         {
21             super.withdrawal(amt);
22
23-             if (getBalance() < MIN_BALANCE)
24             {
25                 System.out.println("Business fee applied: $" + FEE);
26                 super.withdrawal(FEE);
27             }
```

Learning

I learned how inheritance and method overriding work together, and how to reuse code from a parent class. I also learned about object composition using the `Customer` class.

Code reference (toString method):

```
66         @Override
67-         public String toString()
68         {
69             NumberFormat money = NumberFormat.getCurrencyInstance();
70
71             return acctID + "\n" +
72                 cust.toString() +
73                 "Current balance: " + money.format(balance);
74         }
```

